SOFTWARE TESTING

Software Testing Fundamentals

Software Testing is the practice of verifying and validating software to ensure that it meets its specified requirements and behaves as expected.

ISTQB: testing: The process consisting of all lifecycle activities, both static and dynamic, concerned with planning, preparation and evaluation of a component or system and related work products to determine that they satisfy specified requirements, to demonstrate that they are fit for purpose and to detect defects.

Software testing can be divided into two steps:

1.Verification: it refers to the set of tasks that ensure that the software correctly implements a specific function.

2. Validation: it refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements.

Verification: “Are we building the product right?”

Validation: “Are we building the right product?”

What are different types of software testing?

Software Testing can be broadly classified into two types:

1. Manual Testing: Manual testing includes testing software manually, i.e., without using any automation tool or any script. In this type, the tester takes over the role of an end-user and tests the software to identify any unexpected behavior or bug. There are different stages for manual testing such as unit testing, integration testing, system testing, and user acceptance testing.

Testers use test plans, test cases, or test scenarios to test software to ensure the completeness of testing. Manual testing also includes exploratory testing, as testers explore the software to identify errors in it.

2. Automation Testing: Automation testing, which is also known as Test Automation, is when the tester writes scripts and uses another software to test the product. This process involves the automation of a manual process. Automation Testing is used to re-run the test scenarios quickly and repeatedly, that were performed manually in manual testing.

Why Software Testing?

The main reasons for Software Testing are:

- Ensuring Quality: Software Testing helps in ensuring that the software meets quality standards (which is mainly conformance to requirements and expectations).
- Enhancing User Experience: Software Testing helps in improving user experience by identifying usability and functionality issues in the software.
- Compliance: Software Testing helps in ensuring that the software complies with standards and regulations of the specific industry and reduces the risk of legal issues and financial penalties.
- Cost Savings: Software Testing helps in avoiding or lowering costly rework, redesign, and maintenance by identifying defects early in the development process.
- Reputation: Software Testing helps in enhancing the company's reputation by avoiding embarrassing defects and ensuring high quality software.

Consequences of NOT Testing Software

Not testing software properly can have the following consequences:

- Delay / Loss of time
- Futility / Loss of effort
- Wastage / Loss of money
- Shame / Loss of business reputation
- Injury / Death

Software Testing Goals/Objectives

The three main goals of Software Testing are:

- Defect Detection: Find defects/bugs in the software during all stages of its development (earlier, the better).
- Defect Prevention: As a consequence of defect detection, help anticipate and prevent defects from occurring at later stages of development or from recurring in the future.
- User Satisfaction: Ensure customers/users are satisfied that their requirements (explicit or implicit) are met.

Principles of Software Testing

- Testing shows presence of defects
- Exhaustive testing is not possible
- Early testing
- Defect clustering
- Pesticide paradox
- Testing is context dependent
- Absence of errors fallacy

What are the different types of Software Testing Techniques ?

Software testing techniques can be majorly classified into two categories:

1.Black Box Testing: The technique of testing in which the tester doesn't have access to the source code of the software and is conducted at the software interface without any concern with the internal logical structure of the software is known as black-box testing.

2.White-Box Testing: The technique of testing in which the tester is aware of the internal workings of the product, has access to its source code, and is conducted by making sure that all internal operations are performed according to the specifications is known as white box testing.

Black Box Testing	White Box Testing
Internal workings of an application are not required.	Knowledge of the internal workings is a must.
Also known as closed box/data-driven testing.	Also known as clear box/structural testing.
End users, testers, and developers.	Normally done by testers and developers.
This can only be done by a trial and error method.	Data domains and internal boundaries can be better tested.

What are different levels of software testing?

Software level testing can be majorly classified into 4 levels:

1.Unit Testing: A level of the software testing process where individual units/components of a software/system are tested. The purpose is to validate that each unit of the software performs as designed.

2.Integration Testing: A level of the software testing process where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units.

3. System Testing: A level of the software testing process where a complete, integrated system/software is tested. The purpose of this test is to evaluate the system's compliance with the specified requirements.

4. **Acceptance Testing**: A level of the software testing process where a system is tested for acceptability. The purpose of this test is to evaluate the system's compliance with the business requirements and assess whether it is acceptable for delivery.

Software Testing Strategies

The overall strategy for testing includes:

- Before testing starts, it's necessary to identify and specify the requirements of the product in a quantifiable manner.
- Specifying the objectives of testing in a clear and detailed manner.
- For the software, identifying the user's category and developing a profile for each user.

- Developing a test plan to give value and focus on rapid-cycle testing.
- Robust software is developed that is designed to test itself.
- Before testing, using effective formal reviews as a filter
- Conduct formal technical reviews to evaluate the nature, quality or ability of the test strategy and test cases.
- For the testing process, developing a approach for the continuous development.

- Black Box Testing

Black Box Testing is a software testing method in which the functionalities of software applications are tested without having knowledge of internal code structure, implementation details and internal paths. Black Box Testing mainly focuses on input and output of software applications and it is entirely based on software requirements and specifications. It is also known as Behavioral Testing.



Generic steps of black box testing

- The black box test is based on the specification of requirements, so it is examined in the beginning.
- In the second step, the tester creates a positive test scenario and an adverse test scenario by selecting valid and invalid input values to check that the software is processing them correctly or incorrectly.
- In the third step, the tester develops various test cases such as decision table, all pairs test, equivalent division, error estimation, cause-effect graph, etc.
- The fourth phase includes the execution of all test cases.
- In the fifth step, the tester compares the expected output against the actual output.
- In the sixth and final step, if there is any flaw in the software, then it is cured and tested again.

Types of Black Box Testing

- **Functional Testing**

Functional testing is defined as a type of testing that verifies that each function of the software application works in conformance with the requirement and specification.
This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated. It determines the system's software functional requirements.

- **Regression Testing**

Regression Testing is the process of testing the modified parts of the code and the parts that might get affected due to the modifications to ensure that no new errors have been introduced in the software after the modifications have been made.

Regression means the return of something and in the software field, it refers to the return of a bug. It ensures that the newly added code is compatible with the existing code.

In other words, a new software update has no impact on the functionality of the software. This is carried out after a system maintenance operation and upgrades.

- **Nonfunctional Testing**

Non-functional testing is a software testing technique that checks the non-functional attributes of the system.

Non-functional testing is defined as a type of software testing to check non-functional aspects of a software application.

It is designed to test the readiness of a system as per nonfunctional parameters which are never addressed by functional testing.

Non-functional testing is as important as functional testing.

Non-functional testing is also known as NFT. This testing is not functional testing of software. It focuses on the software's performance, usability, and scalability.

Black Box Testing Techniques

Testers use various black box testing techniques to increase test coverage while minimizing the total number of test cases. These techniques are flexible and can be used across all testing levels, including system, unit, integration, functional, and non-functional testing.

1. Equivalence Partitioning

Equivalence partitioning is a black box testing technique that divides input data into distinct partitions or classes where each class is expected to produce similar results. Equivalence classes are created based on requirements and can include factors like data types, ranges, and input relationships. Make sure you include invalid classes as well to test negative scenarios thoroughly.

The primary idea is that if a system handles one value from a partition correctly, it is likely to handle all other values from that partition correctly. This technique reduces the number of test cases needed by testing only one value from each partition. For example:

Consider a software application that accepts an OTP (One-Time Password) consisting of exactly six digits. Any input with fewer or more than six digits will be rejected, and the user will be redirected to an error page.

Valid Partition: Inputs that are exactly six digits long.

Invalid Partitions: Inputs that are fewer than six digits (e.g., four or five digits) and inputs that are more than six digits (e.g., seven, eight, or ten digits).

Equivalence Partitioning Classes		
Invalid	Valid	Invalid
Digits > 6	Digits = 6	Digits < 6

2. Boundary Value Analysis

Boundary value analysis (BVA) is a testing technique focused on the values at the edges of input ranges. It is based on the principle that errors often occur at the boundaries of input ranges rather than within the middle of the range. This technique involves testing the minimum and maximum values of the input range, as well as values just outside these boundaries. Concentrating on these boundary values allows us to identify potential issues that might not be apparent when testing only with values inside the range. For example:

Consider a software application that requires users to enter their age. Applications may contain specific minimum and maximum age restrictions. The restriction is to enter an age between 18 and 30. If the user enters values above 30 or below 18, the application may not work as expected. In this scenario, we can use BVA to test our application by choosing values equal to, above, or below the age limit.

3. Decision Table Testing

Decision table testing is a structured technique to test combinations of inputs and their corresponding outputs based on business rules or logic. A decision table helps represent complex logic by mapping different conditions and actions into a table format. Each table row represents a unique combination of conditions, and the associated expected results are specified. This technique ensures all possible combinations of inputs are tested to verify that the system behaves as expected under various conditions. It is particularly useful for testing systems with complex decision-making logic, as it provides a clear and organized way to design test cases. For example: Let's consider an example where this technique can be used: a discount system that calculates the percentage of discount based on the membership status of the user and purchase amount. In this example, the membership status and the purchase amount are inputs, and the expected discount percentage is the output. You can use this table to ensure the system applies the correct discount for each combination of membership status and purchase amount.

Membership Status	Purchase Amount	Expected Discount
Member	Greater than $100	20%
Member	$100 or less	10%
Non-member	Greater than $100	5%
Non-member	$100 or less	0%

5. Exploratory Testing

Exploratory testing is an informal testing approach in which testers actively explore the application without predefined test cases. It flexibly combines learning, test design, and test execution. This technique detects usability, user experience, or unexpected behavior issues. Exploratory testing allows testers to adapt and focus on areas of the application that seem error-prone or critical. For example:

Suppose we need to test a new website where people buy headphones. Start by exploring the site as a regular user would. Browse the headphones, pick a pair, and add them to the cart. Next, test all the cart functions: adding more items, changing quantities, removing items, and proceeding to checkout. In exploratory testing, testers use their understanding of the product and user behavior to guide the testing. The outcomes of each test inform the next steps.

- ## Cause-Effect Graph

Cause-effect graph comes under the black box testing technique which underlines the relationship between a given result and all the factors affecting the result. It is used to write dynamic test cases. is a technique in which a graph is used to represent the situations of combinations of input conditions. The graph is then converted to a decision table to obtain the test cases. Cause-effect graphing technique is used because boundary value analysis and equivalence class partitioning methods do not consider the combinations of input conditions. But since there may be some critical behaviour to be tested when some combinations of input conditions are considered, that is why cause-effect graphing technique is used.

Advantages of Black Box Testing:

- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used in finding the ambiguity and contradictions in the functional specifications.

Disadvantages of Black Box Testing:

- There is a possibility of repeating the same tests while implementing the testing process.
- Without clear functional specifications, test cases are difficult to implement.
- It is difficult to execute the test cases because of complex inputs at different stages of testing.
- Sometimes, the reason for the test failure cannot be detected.
- Some programs in the application are not tested.
- It does not reveal the errors in the control structure.
- Working with a large sample space of inputs can be exhaustive and consumes a lot of time.

White Box Testing

White Box Testing is a testing technique in which software's internal structure, design, and coding are tested to verify input-output flow and improve design, usability, and security. In white box testing, code is visible to testers, so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing, and Glass box testing.

It is one of two parts of the Box Testing approach to software testing. Its counterpart, Blackbox testing, involves testing from an external or end-user perspective. On the other hand, White box testing in software engineering is based on the inner workings of an application and revolves around internal testing.



Generic steps of white box testing

- Design all test scenarios, test cases and prioritize them according to high priority number.
- This step involves the study of code at runtime to examine the resource utilization, not accessed areas of the code, time taken by various methods and operations and so on.
- In this step testing of internal subroutines takes place. Internal subroutines such as nonpublic methods, interfaces are able to handle all types of data appropriately or not.
- This step focuses on testing of control statements like loops and conditional statements to check the efficiency and accuracy for different data inputs.
- In the last step white box testing includes security testing to check all possible security loopholes by looking at how the code handles security.

What do you verify in White Box Testing?

White box testing involves the testing of the software code for the following:

- Internal security holes
- Broken or poorly structured paths in the coding processes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object, and function on an individual basis
- The testing can be done at system, integration, and unit levels of software development. One of the basic goals of whitebox testing is to verify a working flow for an application. It involves testing a series of predefined inputs against expected or desired outputs so that when a specific input does not result in the expected output, you have encountered a bug.

Types of White Box Testing

White box testing can be done for different purposes. The three main types are:

Unit Testing

- Checks if each part or function of the application works correctly.
- Ensures the application meets design requirements during development.

Integration Testing

- Examines how different parts of the application work together.
- Done after unit testing to make sure components work well both alone and together.

Regression Testing

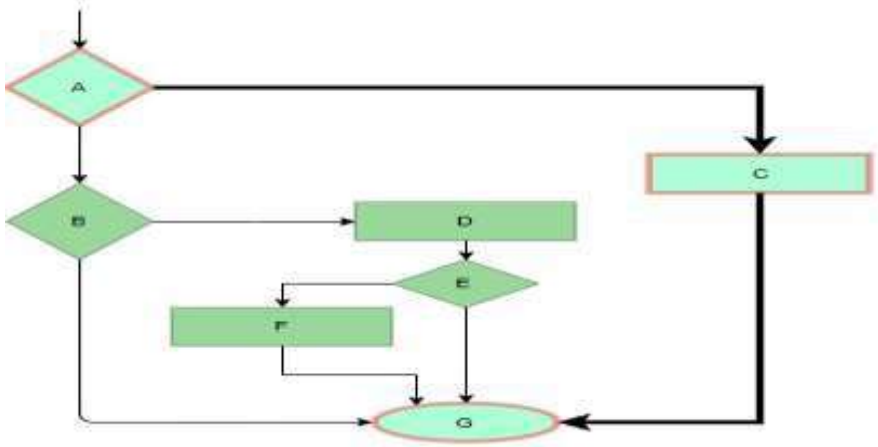
- Verifies that changes or updates don’t break existing functionality.
- Ensures the application still passes all existing tests after updates.

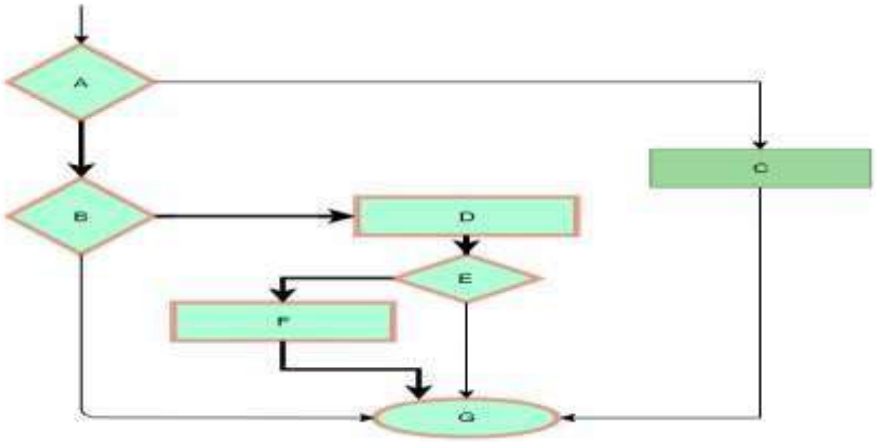
- **White Box Testing Techniques**

One of the main benefits of white box testing is that it allows for testing every part of an application. To achieve complete code coverage, white box testing uses the following techniques:

1. Statement Coverage

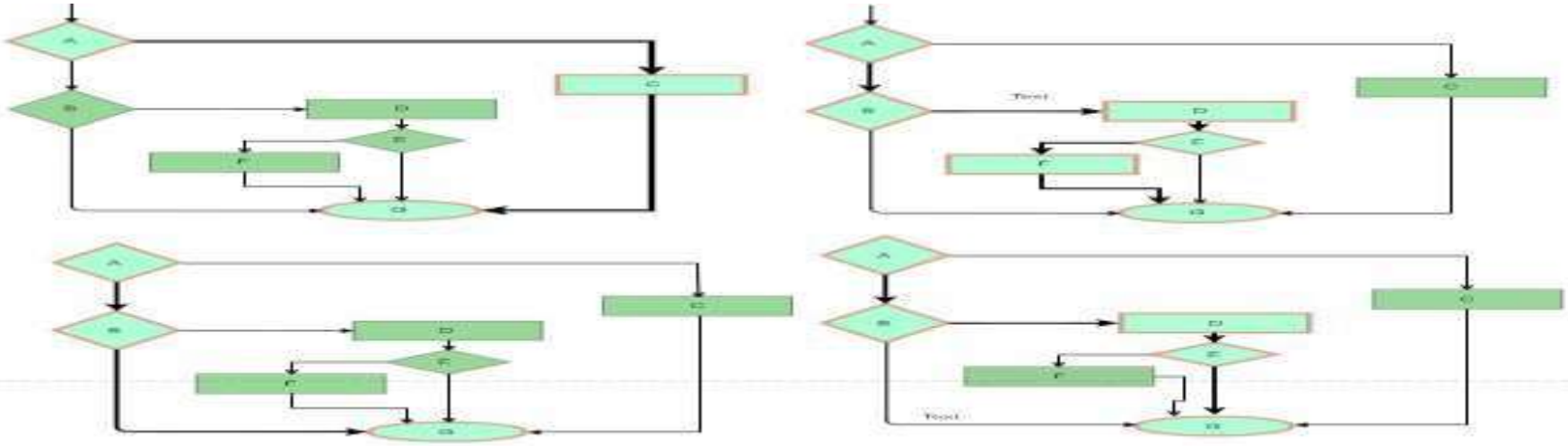
In this technique, the aim is to traverse all statements at least once. Hence, each line of code is tested. In the case of a flowchart, every node must be traversed at least once. Since all lines of code are covered, it helps in pointing out faulty code.





2. Branch Coverage

In this technique, test cases are designed so that each branch from all decision points is traversed at least once. In a flowchart, all edges must be traversed at least once.



4 test cases are required such that all branches of all decisions are covered, i.e, all edges of the flowchart are covered

3. Condition Coverage

In this technique, all individual conditions must be covered as shown in the following example:

- READ X, Y
- IF(X == 0 || Y == 0)
- PRINT '0'
- #TC1 – X = 0, Y = 55
- #TC2 – X = 5, Y = 0

4. Multiple Condition Coverage

In this technique, all the possible combinations of the possible outcomes of conditions are tested at least once. Let's consider the following example:

- READ X, Y
- IF(X == 0 || Y == 0)
- PRINT '0'
- #TC1: X = 0, Y = 0
- #TC2: X = 0, Y = 5
- #TC3: X = 55, Y = 0
- #TC4: X = 55, Y = 5

5. Basis Path Testing

In this technique, control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each independent path. **Steps:**

- Make the corresponding control flow graph
- Calculate the cyclomatic complexity
- Find the independent paths
- Design test cases corresponding to each independent path
- V(G) = P + 1, where P is the number of predicate nodes in the flow graph
- V(G) = E – N + 2, where E is the number of edges and N is the total number of nodes
- V(G) = Number of non-overlapping regions in the graph
- #P1: 1 – 2 – 4 – 7 – 8
- #P2: 1 – 2 – 3 – 5 – 7 – 8
- #P3: 1 – 2 – 3 – 6 – 7 – 8
- #P4: 1 – 2 – 4 – 7 – 1 – . . . – 7 – 8

6. Loop Testing

Loops are widely used and these are fundamental to many algorithms hence, their testing is very important. Errors often occur at the beginnings and ends of loops.

- **Simple loops:** For simple loops of size n, test cases are designed that:
  1. Skip the loop entirely
  2. Only one pass through the loop
  3. 2 passes
  4. m passes, where m < n
  5. n-1 ans n+1 passes
- **Nested loops:** For nested loops, all the loops are set to their minimum count, and we start from the innermost loop. Simple loop tests are conducted for the innermost loop and this is worked outwards till all the loops have been tested.
- **Concatenated loops:** Independent loops, one after another. Simple loop tests are applied for each. If they're not independent, treat them like nesting.

Features of white box testing:

- **Code coverage analysis:**

White box testing helps to analyze the code coverage of an application, which helps to identify the areas of the code that are not being tested.

- **Access to the source code:**

White box testing requires access to the application's source code, which makes it possible to test individual functions, methods, and modules.

- **Knowledge of programming languages:**

Testers performing white box testing must have knowledge of programming languages like Java, C++, Python, and PHP to understand the code structure and write tests.

- **Identifying logical errors:**

White box testing helps to identify logical errors in the code, such as infinite loops or incorrect conditional statements.

- **Integration testing:**

White box testing is useful for integration testing, as it allows testers to verify that the different components of an application are working together as expected.

- **Unit testing:**

White box testing is also used for unit testing, which involves testing individual units of code to ensure that they are working correctly.

- **Optimization of code:**

White box testing can help to optimize the code by identifying any performance issues, redundant code, or other areas that can be improved.

- **Security testing:**

White box testing can also be used for security testing, as it allows testers to identify any vulnerabilities in the application's code.

Reasons for white box testing

- It identifies internal security holes.
- To check the way of input inside the code.
- Check the functionality of conditional loops.
- To test function, object, and statement at an individual level.

Advantages of White box testing

- White box testing optimizes code so hidden errors can be identified.
- Test cases of white box testing can be easily automated.
- This testing is more thorough than other testing approaches as it covers all code paths.
- It can be started in the SDLC phase even without GUI.

Disadvantages of White box testing

- White box testing is too much time consuming when it comes to large-scale programming applications.
- White box testing is much expensive and complex.
- It can lead to production error because it is not detailed by the developers.
- White box testing needs professional programmers who have a detailed knowledge and understanding of programming language and implementation.

Difference between white-box testing and black-box testing

White-box testing	Black box testing
The developers can perform white box testing.	The test engineers perform the black box testing.
To perform WBT, we should have an understanding of the programming languages.	To perform BBT, there is no need to have an understanding of the programming languages.
In this, we will look into the source code and test the logic of the code.	In this, we will verify the functionality of the application based on the requirement specification.
In this, the developer should know about the internal design of the code.	In this, there is no need to know about the internal design of the code.

i

Basis Path Testing

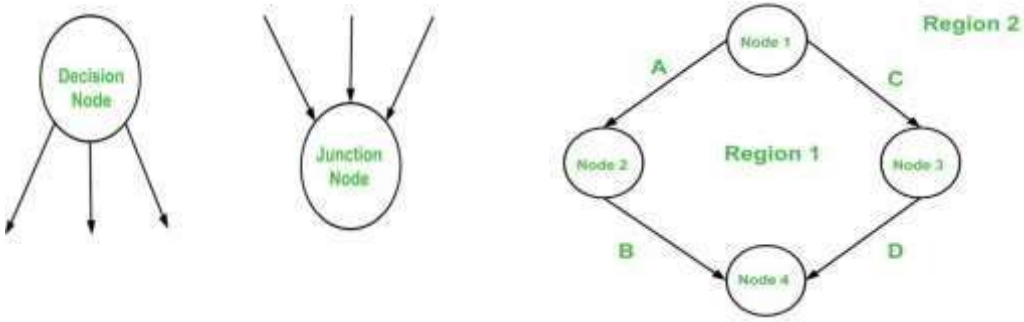
Basis Path Testing is a white-box testing technique based on the control structure of a program or a module. Using this structure, a control flow graph is prepared and the various possible paths present in the graph are executed as a part of testing. Therefore, Basis path testing is a technique of selecting the paths in the control flow graph, that provide a basis set of execution paths through the program or module. Since this testing is based on the control structure of the program, it requires complete knowledge of the program's structure.

To design test cases using this technique, four steps are followed :

- Construct the Control Flow Graph
- Compute the Cyclomatic Complexity of the Graph
- Identify the Independent Paths
- Design Test cases from Independent Paths

Control Flow Graph – A control flow graph (or simply, flow graph) is a directed graph which represents the control structure of a program or module. A control flow graph (V, E) has V number of nodes/vertices and E number of edges in it. A control graph can also have :

- Junction Node – a node with more than one arrow entering it.
- Decision Node – a node with more than one arrow leaving it.
- Region – area bounded by edges and nodes (area outside the graph is also counted as a region.).



Cyclomatic Complexity – The cyclomatic complexity V(G) is said to be a measure of the logical complexity of a program. It can be calculated using three different formulae :

Formula based on edges and nodes :

V(G) = e - n + 2*P

Where, e is number of edges, n is number of vertices, P is number of connected components.

Independent Paths : An independent path in the control flow graph is the one which introduces at least one new edge that has not been traversed before the path is defined. The cyclomatic complexity gives the number of independent paths present in a flow graph.

Advantages : Basis Path Testing can be applicable in the following cases:

- **More Coverage –** Basis path testing provides the best code coverage as it aims to achieve maximum logic coverage instead of maximum path coverage. This results in an overall thorough testing of the code.
- **Maintenance Testing –** When a software is modified, it is still necessary to test the changes made in the software which as a result, requires path testing.
- **Unit Testing –** When a developer writes the code, he or she tests the structure of the program or module themselves first. This is why basis path testing requires enough knowledge about the structure of the code.

- **Integration Testing –** When one module calls other modules, there are high chances of Interface errors. In order to avoid the case of such errors, path testing is performed to test all the paths on the interfaces of the modules.

- **Testing Effort –** Since the basis path testing technique takes into account the complexity of the software (i.e., program or module) while computing the cyclomatic complexity, therefore it is intuitive to note that testing effort in case of basis path testing is directly proportional to the complexity of the software or program.